ChatGPT

Compute-Matched Offline RL Robustness: Decision Transformer vs IQL on D4RL

Executive summary

This report proposes a **feasible, research-oriented deep learning project** that fits a typical **8–10 week** group-project window and satisfies the proposal expectations in your uploaded **proposal rubric** (clear motivation, literature gap, testable question, rigorous methodology, reproducibility) and the course **project guidance** (relevance, feasibility, novelty, measurability, proper multi-seed evaluation, diagnostics and interpretation) .

The recommended project investigates a clear, modern DL/RL question explicitly suggested by the guidance's "modern DL in RL" theme (transformer-style sequence models in RL) :

Can transformer-based sequence modeling (Decision Transformer) be more robust than value-based offline RL (IQL) under matched compute, across dataset quality regimes? [1]

Why this is a strong project (analytically and strategically):
- It is **methodologically clean**: offline training is from a fixed dataset (reducing confounds), and evaluation can be standardized and compute-matched across methods. [2]
- It is **scientifically relevant**: offline RL is motivated by settings where online data collection is costly/unsafe, while distribution shift and evaluation reliability are core open challenges. [3]
- It is **measurable and reproducible**: you can run **≥10 seeds per condition** and report uncertainty-aware metrics, aligning with the rubric's requirement for statistical reporting and reproducibility. [4]

The core deliverable is a **reproducible empirical study** that clarifies **when/why** each approach works (dataset-quality dependence, compute scaling, seed sensitivity) rather than a vague "X is better than Y," aligning directly with your rubric's recommended framing.

Project definition and motivation

Project title and concise motivation

Title (≤15 words): Compute-Matched Offline RL Robustness: Decision Transformer vs IQL on D4RL

Motivation (rubric-aligned, concrete):
Your proposal rubric explicitly asks for a **concrete outcome** and warns against generic claims, recommending a measurable target (example: "≥20% sample complexity reduction").
Offline RL is compelling because it learns from static datasets (no online interaction), but it is notoriously

sensitive to **distributional shift** (learning a policy that deviates from the data) and to evaluation practices that can mask variance. [5]

Concrete project outcome:
Produce a reproducible benchmark-style comparison showing (with confidence intervals and ≥10 seeds) which method is **more robust** under: - **Dataset quality regimes** (e.g., *medium*, *medium-replay*, *medium-expert*). [6]
- **Compute constraints** (fixed gradient updates / wall-clock budget).
- **Matched model capacity** (roughly matched parameter counts) to prevent "bigger model wins" confounds highlighted in generalization benchmarks. [7]

What's the gap in the literature?

- **Decision Transformer (DT)** frames RL as conditional sequence modeling with a causal Transformer conditioned on desired return, and reports strong offline performance on multiple benchmarks. [8]
- **IQL** aims to avoid out-of-dataset action evaluation by using expectile regression for value learning and advantage-weighted behavioral cloning for policy extraction, reporting state-of-the-art results on D4RL. [9]
- Evaluation in deep RL/offline RL is widely recognized as fragile without careful multi-run statistics and uncertainty reporting. [10]

Gap to target: A **compute-matched, seed-robust comparative study** that quantifies:
1) **Seed sensitivity** and uncertainty-aware performance differences between DT and IQL (and optionally CQL/BC), and
2) **Dataset-quality dependence**, especially the known D4RL mixtures like *medium-expert* and replay-buffer style datasets like *medium-replay*. [11]

This aligns with your rubric's request to define a precise, comparable "your angle" (controlled study; shared metrics/datasets).

Candidate project variants

Variant	What you do	Novelty	Feasibility (8–10 weeks)	Compute profile	When to choose
Core (recommended)	DT vs IQL + Behavior Cloning (BC) on a small D4RL slice; matched compute, ≥10 seeds	Medium (rigorous evaluation angle)	High	Moderate	Best tradeoff: rigorous + feasible
Expanded baselines	Add CQL (and maybe BCQ) to represent value-pessimism family	Medium–High	Medium	Higher	If you have stronger compute and want broader conclusions [12]

Variant	What you do	Novelty	Feasibility (8–10 weeks)	Compute profile	When to choose
Generalization add-on	Add “generalization gap” or dataset-shift evaluation protocol (train on one dataset variant, eval on another task variant)	High	Medium	Moderate–High	If you want a more “researchy” angle and can design a clean split [13]

Recommendation: Start with the **Core** variant, and treat “Expanded baselines” as a stretch goal. This is consistent with the guidance to keep environment/task complexity appropriate and deliver a measurable contribution within ~8–10 weeks.

Unspecified details you must confirm for the proposal: your available hardware (GPU model / cluster access), group size, and how many total experiment conditions you can afford while still meeting the rubric’s multi-seed expectation.

Research questions and hypotheses

Your rubric recommends questions of the form **“under which conditions does X outperform Y?”** rather than “prove X is better.”

Primary research question

RQ1: Under a **matched compute budget** and standardized evaluation, how do **Decision Transformer** and **IQL** compare in (a) final performance and (b) seed robustness across D4RL dataset qualities? [14]

Secondary research questions

RQ2: How do conclusions change as dataset quality shifts among D4RL variants (e.g., *medium*, *medium-replay*, *medium-expert*)? [6]

RQ3: Which method dominates in the **compute-limited regime** (e.g., 25%, 50%, 100% of a baseline training budget)? [15]

Testable hypotheses

These are framed as falsifiable statements and explicitly tied to known algorithmic design differences:

H1 (dataset-quality dependence): DT will show stronger gains over BC on higher-quality datasets (e.g., *medium-expert*) than on replay/mixed datasets (e.g., *medium-replay*), where return conditioning can be harder due to multimodal behavior and inconsistent trajectories. [16]

H2 (robustness to distribution shift): IQL will exhibit better robustness than naïve TD methods because it is designed to avoid explicit out-of-dataset action value queries, but its sensitivity to key hyperparameters (expectile τ, advantage temperature β) may still create seed variance under tight compute. [17]

H3 (compute efficiency): Under small compute budgets, IQL will reach competitive performance faster than DT because DT's transformer sequence modeling may require more optimization steps to stabilize representations, while IQL uses relatively direct value/backups and policy extraction. [18]

Experimental methodology and reproducibility

The rubric explicitly asks you to specify setting/data, models, baselines, hyperparameter search, seeds/statistics, metrics/plots, compute plan, and reproducibility practices.

Environments and datasets

Dataset family: D4RL offline datasets (primarily MuJoCo locomotion tasks). D4RL was introduced as a benchmark suite for offline RL and includes standardized datasets and evaluation protocols. [19]

Recommended tasks (keep it feasible):
Select **two** locomotion environments for the final study: - `hopper-*` and `walker2d-*` (or `halfcheetah-*`), because these are common D4RL baselines and keep comparisons interpretable. [20]

Dataset-quality conditions (minimal but informative):
Use **three** dataset types per environment: - `medium`
- `medium-replay` (replay-buffer mixture)
- `medium-expert` (50/50 mixture of medium and expert data) [6]

This 2×3 design supports RQ2 while staying manageable.

Data access choice (state explicitly in proposal):
- Option A: Use the Farama Foundation [21] D4RL repository and loaders. [22]
- Option B (often cleaner today): Use Minari's hosted D4RL datasets for standardized dataset packaging and environment recovery. [23]

If you don't decide immediately, state it as **unspecified** and commit to one by end of Week 1.

Methods and architectures

To keep comparability, match observation preprocessing and network scales as closely as the algorithm families allow.

Decision Transformer (DT): causal Transformer conditioned on return-to-go / past states and actions (sequence modeling view of RL). [8]
- Architecture (suggested starting point): 3–6 transformer blocks, hidden size 128–256, context length 20–50 for low-dimensional MuJoCo states (explicitly treat these as tunables). [24]

Implicit Q-Learning (IQL): value learning with expectile regression and advantage-weighted behavioral cloning. [25]
- Architecture: standard MLP critics (2–3 layers, 256 units) + policy MLP (2–3 layers, 256 units), consistent with common D4RL baselines (declare as tunables if needed). [26]

Behavior Cloning (BC) baseline (required): supervised policy learning on the dataset actions; widely used as a stable offline baseline and often dominates poorly-covered datasets. [2]

Optional strong baseline (stretch): Conservative Q-Learning (CQL) to represent pessimism/ regularization approaches in offline RL. [27]

Implementation choice and starter codebases

Your rubric emphasizes “clean baselines” and reproducibility. A practical way to achieve this is to standardize on one library that already includes robust offline RL implementations.

Recommended single-framework path: Use **d3rlpy**, an offline RL library that supports IQL, CQL, and Decision Transformer, and explicitly aims to help with reproducibility via benchmarked implementations. [28]

Reference/validation repos (for cross-checking):
- Official Decision Transformer codebase. [29]
- Official IQL repo (useful for checking hyperparameter conventions even if you implement elsewhere). [30]
- Official CQL repo. [31]

Hyperparameter search plan

The proposal rubric expects a **defined search space** and a **selection protocol** with a fixed budget.

Principle: Match tuning budget across methods to avoid “the better-tuned method wins.” This aligns with the reproducibility concerns raised in deep RL evaluation work. [32]

Suggested protocol: - Use **1–2 environments** and **1 dataset type** (e.g., `hopper-medium`) as a *development setting* for initial tuning only.
- Freeze hyperparameters, then run the full grid of conditions for final reporting.

Search method: random search or small grid (feasible), with an explicit cap (e.g., 20 trials per algorithm).

Example search spaces (explicitly state you may adjust):

DT: - learning rate: {1e-4, 3e-4, 1e-3}
- weight decay: {0, 1e-4, 1e-3}
- context length K: {20, 30, 50} [24]

IQL: - actor lr / critic lr: {3e-4, 1e-3}
- expectile τ: {0.7, 0.8, 0.9} (core IQL knob) [25]
- advantage temperature β: {1, 3, 10} [33]

BC: - lr: {1e-4, 3e-4, 1e-3}
- weight decay: {0, 1e-4} [34]

Selection rule: best mean validation return across **3 seeds** on the dev setting; then lock and run full experiments with **10 seeds**. The rubric expects 10 seeds for final reporting, so position 3-seed tuning as "pilot," not final evidence.

Compute budget and matched-compute definition

Your rubric suggests matched compute (same wall-clock or steps) and asks you to detail hardware (e.g., cluster vs personal GPU vs Colab).

Unspecified in your request: available hardware.
Below is a **hardware-agnostic plan** that scales:

Per-run training budget (define precisely): - Budget A: 200k gradient steps
- Budget B: 500k gradient steps
- Budget C: 1M gradient steps

This directly enables **RQ3** (compute-limited regime) and keeps comparisons fair across methods.

Operational definition of "matched compute": - Primary: equal **number of optimizer steps** and batch size (compute-proxy).
- Secondary (if feasible): track and match **wall-clock time** per run for DT vs IQL, since transformer forward passes can be costlier. [15]

Reproducibility practices

The rubric explicitly requires pinned versions, deterministic scripts, and a public repo with a clear README plus environment definitions.

Minimum standard (include in proposal and final deliverable): - **Seeds:** 10 seeds per condition; report mean ± standard error; show per-seed traces.
- **Versioning:** `requirements.txt` or `environment.yml` + recorded git commit hash.
- **Deterministic run scripts:** single command reproduces each figure/table (e.g., `python run.py --algo IQL --env hopper --dataset medium --seed 7`).
- **Logging:** store raw per-epoch returns, evaluation returns, and training loss curves to disk (CSV/JSON).
- **Evaluation discipline:** fixed evaluation protocol and evaluation frequency for all methods (e.g., 10 episodes every N updates). [15]

These choices are consistent with reproducibility concerns emphasized in deep RL evaluation scholarship. [32]

Experimental workflow diagram

```
flowchart TD
  A[Select D4RL tasks + dataset variants] --> B[Pick algorithms: DT, IQL, BC (+
optional CQL)]
  B --> C[Implement standard training + eval harness]
  C --> D[Pilot runs: sanity checks, 1-2 seeds]
  D --> E[Hyperparameter tuning on dev setting]
  E --> F[Freeze hyperparameters]
  F --> G[Full runs: all conditions, 10 seeds each]
  G --> H[Aggregate metrics + uncertainty intervals]
  H --> I[Statistical comparisons + ablations]
  I --> J[Figures, tables, reproducibility checklist + code release]
```

Prioritized reading list

This list is designed so that you can (a) implement correctly and (b) evaluate correctly. It prioritizes primary sources and includes recent "evaluation rigor" work.

1) **D4RL: Datasets for Deep Data-Driven Reinforcement Learning (2020)**
Defines the D4RL benchmark suite, explains dataset design factors, and provides standardized evaluation for offline RL—central to this project's datasets and metrics. [35]

2) **Decision Transformer: Reinforcement Learning via Sequence Modeling (2021)**
Introduces the return-conditioned transformer framing of offline RL, treating RL as sequence modeling with a causal transformer; this is the core "transformer baseline" you will evaluate. [8]

3) **Offline Reinforcement Learning with Implicit Q-Learning (2021)**
Presents IQL's expectile-based value learning and advantage-weighted BC extraction, explicitly designed to avoid querying out-of-dataset action values while achieving strong D4RL performance. [25]

4) **Conservative Q-Learning for Offline Reinforcement Learning (2020)**
A foundational pessimism method in offline RL; useful as an optional baseline and as a conceptual contrast to IQL and DT regarding distribution shift. [27]

5) **Offline RL: Tutorial, Review, and Perspectives on Open Problems (2020)**
A tutorial-style paper covering why offline RL is challenging (distribution shift, extrapolation, evaluation), plus a broad view of solution families; excellent for writing your Literature Gap section. [34]

6) **Deep Reinforcement Learning that Matters (2017/2018)**
Classic paper showing how variability, non-determinism, and reporting practices can mislead; provides motivation for your multi-seed, uncertainty-aware evaluation posture. [36]

7) **Deep RL at the Edge of the Statistical Precipice (2021)**
Argues that few-run evaluation is statistically fragile and motivates better uncertainty reporting; pairs well with using robust aggregate metrics and confidence intervals. [37]

8) **RLiable (library + methodology, 2021–)**
Practical toolkit and methodology for reliable RL evaluation (confidence intervals, aggregate metrics such as IQM); highly relevant to your “rigorous analysis” deliverables. [38]

9) **Empirical Design in Reinforcement Learning (2024)**
Modern guidance on empirical design: emphasizes intervals over only hypothesis tests, discusses seed handling and experimental design choices; complements your course rubric’s evaluation expectations. [39]

10) **d3rlpy: An Offline Deep Reinforcement Learning Library (2022, JMLR)**
If you adopt d3rlpy as the main implementation stack, this paper explains design choices and reports large-scale benchmarks intended to improve reproducibility. [40]

Timeline and deliverables

Your rubric expects feasibility, planning, and an explicit timeline; it also notes your proposal must be concise and submitted as a PDF with strict formatting.

Assumption: planning starts **Feb 20, 2026** (today in America/Toronto).
Hard external date: proposal due **Feb 24, 2026** per the rubric.

Week-by-week plan (8–10 weeks)

Week 1 (Feb 20–Feb 26): Proposal + setup
Deliverables: 1-page proposal draft (title, motivation, gap, RQs, methodology), repo skeleton, environment setup and dataset download script. The rubric’s required sections guide the proposal structure.

Week 2: Baseline harness and sanity checks
Deliverables: unified training/eval harness; BC baseline runs on 1 env × 1 dataset; verify metrics logging and deterministic seeds (at least locally). [41]

Week 3: Implement/enable DT and IQL
Deliverables: DT and IQL running end-to-end on the dev setting, producing learning curves and final scores. Validate on small budgets first. [42]

Week 4: Hyperparameter tuning (bounded budget)
Deliverables: fixed HP set per algorithm, with logs and selection justification. Use 3 seeds in dev setting; document total trials.

Week 5: Full-scale runs (main conditions)
Deliverables: start 10-seed runs for the core matrix (2 envs × 3 dataset types × 3 algos). Store raw results as artifact.

Week 6: Analysis infrastructure + mid-project checkpoint
Deliverables: analysis notebooks/scripts for aggregation, confidence intervals, rliable integration, and standardized plots. [43]

Week 7: Compute-budget sweep (RQ3)
Deliverables: repeat key conditions under reduced compute budgets (e.g., 25/50/100%) to study compute sensitivity. [15]

Week 8: Robustness diagnostics and ablations
Deliverables: interpretability/diagnostics: sensitivity plots over τ/β (IQL) and context length (DT), plus failure case analysis by dataset type. [44]

Week 9: Writing and reproducibility packaging
Deliverables: full report draft; reproducibility checklist; replication instructions; final plots/tables locked. [32]

Week 10 (buffer): polish, reruns, and final submission
Deliverables: final report PDF, finalized GitHub release tag, "one-command reproduce" scripts where possible.

Mermaid timeline

```
gantt
  title 8–10 Week Offline RL Project Timeline
  dateFormat  YYYY-MM-DD
  section Proposal & Setup
  Proposal draft + repo skeleton :active, a1, 2026-02-20, 7d
  section Implementation
  Baseline harness + BC sanity checks :a2, 2026-02-27, 7d
  DT + IQL end-to-end runs :a3, 2026-03-06, 7d
  section Tuning & Experiments
  Hyperparameter tuning (bounded) :a4, 2026-03-13, 7d
  Full runs (10 seeds, main grid) :a5, 2026-03-20, 14d
  section Analysis & Writing
  rliable + plots + diagnostics :a6, 2026-04-03, 14d
  Compute-budget sweep + ablations :a7, 2026-04-17, 14d
  Report + reproducibility package :a8, 2026-05-01, 14d
```

Tooling, datasets, and starter repositories

This section lists the practical stack you'll need and links to official sources where possible.

Datasets and data tooling

- **D4RL benchmark** datasets and environments. [45]

- **Minari** (optional but recommended if you want standardized dataset packaging and environment recovery APIs). [46]
- D4RL task definitions (e.g., meaning of *medium-replay* and *medium-expert*). [47]

Core libraries and codebases

Recommended primary implementation (single-stack): - **d3rlpy** (supports IQL, CQL, and Decision Transformer in a unified API; includes benchmarking focus). [48]

Official reference repos (for validation and transparency): - Decision Transformer official repo. [29]
- IQL official repo. [30]
- CQL official repo. [31]

Evaluation tooling: - rliable for uncertainty-aware evaluation and aggregate metrics. [49]

Minimal starter-repo plan

A strong "starter repo" layout for reproducibility (matches rubric expectations) :

- `configs/` (YAML configs per algorithm and environment)
- `scripts/`
- `download_data.sh` (or Python equivalent for Minari/D4RL) [50]
- `train.py` (single entrypoint)
- `eval.py`
- `sweep.py` (bounded HP sweeps)
- `results/` (excluded from git; stored as artifacts)
- `analysis/` (one notebook to reproduce each figure; or a single `make_figures.py`)
- `environment.yml` or `requirements.txt`

Evaluation plan, statistics, and visualization

Your guidance explicitly expects proper evaluation with multiple seeds and statistically meaningful results, plus diagnostics and learning curves.

Metrics

Primary metric: D4RL normalized score (standard offline RL benchmark reporting). [51]

Secondary metrics (robustness/learning dynamics): - Mean episodic return (raw) and normalized score both, to ensure interpretability. [52]
- Compute-efficiency curves: performance vs gradient steps (RQ3). [15]
- Seed robustness: variance/standard deviation across seeds; interquartile range; probability of outperforming BC. [53]

Statistical tests and uncertainty reporting

Reliable evaluation in deep RL often requires **uncertainty-aware reporting**, not just point estimates. [54]

Recommended approach: - Report **mean ± standard error** (as your rubric specifies) and show individual seed traces.
- Add **bootstrap confidence intervals** for aggregate metrics (e.g., mean or IQM), and use rliable's recommended summaries for RL benchmarks. [55]
- For pairwise comparisons (DT vs IQL), prefer: - bootstrap CI of the difference in aggregate performance, and/or
- permutation tests if you want a formal p-value (document the choice and assumptions). [56]

Visualizations to include

To satisfy "metrics & plots" requirements and enable interpretation rather than "it works," produce:

- **Learning curves**: normalized score vs training steps, with mean curve + shaded SE, plus faint per-seed traces.
- **Violin/box plots**: final performance distribution across seeds for each method and dataset type. [15]
- **Compute-scaling plot**: 25/50/100% compute budget vs final performance (one plot per environment).
- **Sensitivity heatmaps**:
- IQL: τ × β grid vs performance (to show stability regions). [25]
- DT: context length × model size vs performance. [24]

Tables to include (requested by you): - Table: all experiment conditions (env, dataset type, algorithm, compute budget, seeds).
- Table: final metrics summary (mean ± SE + CI) per condition.

Risks, mitigations, and success criteria

Your rubric requires at least one risk and a fallback plan.

Key risks and mitigations

Risk: compute blow-up from too many conditions while meeting 10 seeds/condition.
Mitigation: restrict final scope to **2 environments × 3 dataset types × 3 algorithms**, and treat extras (CQL, extra envs) as stretch goals. Pilot with fewer seeds and expand only for final results.

Risk: unfair comparisons due to unequal tuning effort.
Mitigation: enforce a fixed hyperparameter tuning budget per method, and document the budget and dev setting. [32]

Risk: "library default" results do not match reported baselines.
Mitigation: validate your pipeline by reproducing at least one known baseline curve or sanity check using an official repo or a published benchmark. [57]

Risk: conclusions are statistically ambiguous (overlapping uncertainty).
Mitigation: adopt uncertainty-aware evaluation (rliable, bootstrap CIs, IQM) and interpret results in terms of effect sizes and intervals rather than binary claims. [4]

Success criteria (clear and measurable)

A reasonable set of success criteria consistent with the rubric's "concrete metric" guidance :

- You produce a fully reproducible experiment suite where a third party can re-run core experiments with one command and regenerate all plots/tables.
- You report 10-seed results for the core grid and include uncertainty intervals and per-seed traces. [53]
- You identify at least one **clear regime-dependent conclusion**, e.g.: "DT dominates on medium-expert but loses on medium-replay under compute limit B," supported by effect sizes and CIs. [58]
- You provide at least one diagnostic insight (hyperparameter sensitivity map or compute-scaling behavior) explaining *why* differences occur, consistent with the guidance's "go beyond it works/ doesn't work."

Recommended final deliverable format

To align with the rubric's emphasis on writing quality and reproducibility :

- **Final report (PDF)** with: Motivation → Gap → RQs/Hypotheses → Methodology → Results (with uncertainty) → Diagnostics/Interpretation → Limitations/Future work.
- **Code release** (public repo) with pinned dependencies, reproducible run scripts, and raw result artifacts.
- **Reproducibility checklist** including: exact dataset IDs, seeds list, machine specs, total compute, git commit hashes, and "how to reproduce Figure X" commands. [32]

---

[1] [8] [14] [16] [18] [24] [42] Decision Transformer: Reinforcement Learning via Sequence Modeling
https://arxiv.org/abs/2106.01345?utm_source=chatgpt.com

[2] [19] [20] [21] [35] [45] [51] [52] D4RL: Datasets for Deep Data-Driven Reinforcement Learning
https://arxiv.org/abs/2004.07219?utm_source=chatgpt.com

[3] [5] [34] Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems
https://arxiv.org/abs/2005.01643?utm_source=chatgpt.com

[4] [15] [37] [53] [54] Deep Reinforcement Learning at the Edge of the Statistical ...
https://arxiv.org/abs/2108.13264?utm_source=chatgpt.com

[6] [11] [47] [58] Tasks · Farama-Foundation/D4RL Wiki
https://github.com/Farama-Foundation/d4rl/wiki/Tasks?utm_source=chatgpt.com

[7] Leveraging Procedural Generation to Benchmark Reinforcement Learning
https://arxiv.org/abs/1912.01588?utm_source=chatgpt.com

[9] [17] [25] [26] [33] [44] Offline Reinforcement Learning with Implicit Q-Learning
https://arxiv.org/abs/2110.06169?utm_source=chatgpt.com

10 32 36 Deep Reinforcement Learning that Matters
https://arxiv.org/abs/1709.06560?utm_source=chatgpt.com

12 27 Conservative Q-Learning for Offline Reinforcement Learning
https://arxiv.org/abs/2006.04779?utm_source=chatgpt.com

13 THE GENERALIZATION GAP IN OFFLINE ...
https://proceedings.iclr.cc/paper_files/paper/2024/file/5c1ddd2e59df46fd2aa85c833b1b36ed-Paper-Conference.pdf?utm_source=chatgpt.com

22 Farama-Foundation/D4RL
https://github.com/Farama-Foundation/D4RL?utm_source=chatgpt.com

23 46 Farama-Foundation/Minari: A standard format for offline ...
https://github.com/Farama-Foundation/Minari?utm_source=chatgpt.com

28 40 48 d3rlpy: An Offline Deep Reinforcement Learning Library
https://www.jmlr.org/papers/v23/22-0017.html?utm_source=chatgpt.com

29 57 Official codebase for Decision Transformer
https://github.com/kzl/decision-transformer?utm_source=chatgpt.com

30 ikostrikov/implicit_q_learning
https://github.com/ikostrikov/implicit_q_learning?utm_source=chatgpt.com

31 aviralkumar2907/CQL: Code for conservative Q-learning
https://github.com/aviralkumar2907/CQL?utm_source=chatgpt.com

38 Deep RL at the Edge of the Statistical Precipice
https://agarwl.github.io/rliable/?utm_source=chatgpt.com

39 41 56 Empirical Design in Reinforcement Learning
https://jmlr.org/papers/volume25/23-0183/23-0183.pdf?utm_source=chatgpt.com

43 49 55 google-research/rliable
https://github.com/google-research/rliable?utm_source=chatgpt.com

50 Minari Documentation
https://minari.farama.org/index.html?utm_source=chatgpt.com